Tema 4
Verificación y validación

Pruebas de software

Contenidos

1. Motivación.
2. Testing.
3. Pruebas de caja negra.
4. Pruebas de caja blanca.
5. Herramientas y buenas prácticas para el testing.

1. Motivación

¿Qué es la verificación y validación de software?

1. Motivación – Calidad en el software

El software que escribimos contiene errores.

🎯 **Objetivo de la calidad en el Software:**
Entregar software sin defectos.

- Para conseguir calidad, no solo necesitamos testing.
- Pero el testing es fundamental.

> Quality is not equal to test. Quality is achieved by putting development and testing into a blender and mixing them until one is indistinguishable from the other.
>
> Testing must be an unavoidable aspect of development, and the marriage of development and testing is where quality is achieved.

"How Google Tests Software", James Whittaker Jason Arbon y Jeff Carollo, Addison-Wesley, 2012

1. Motivación – Terminología

Error

(humano)

- La causa de un defecto.
- Un error de diseño, un error de implementación, una errata, etc.

→ **Defecto**

(código)

- Se refiere a la parte del código que es incorrecta.
- Condición necesaria, pero no suficiente para que haya un fallo.

```
if (false)
  x = 1/0
```

→ **Fallo**

(comportamiento observable)

- Comportamiento incorrecto del software.
- Se manifiesta cuando el sistema está en funcionamiento.

1. Motivación – Ejercicio

```
int doblarValor(int i) {
  int resultado = i * i;
  return resultado;
}
```

La llamada a `doblarValor(3)` devuelve `9`. Esto es:

- ☐ Un fallo.
- ☐ Un defecto.
- ☐ Un error.

1. Motivación – Ejercicio

```
int doblarValor(int i) {
  int resultado = i * i;
  return resultado;
}
```

La llamada a `doblarValor(3)` devuelve 9. Esto es:

- [x] Un fallo.
- [ ] Un defecto.
- [ ] Un error.

Es un fallo porque es un comportamiento incorrecto del programa.
Al hacer la llamada todavía no sabemos dónde está el defecto.

DUDAS:
¿En qué línea está el defecto? ¿En qué línea está el error?

1. Motivación – Verificación y validación

Verificación
¿Se está construyendo el producto correctamente?

Validación
¿Se está construyendo el producto correcto?

Diferencia es sutil pero importante:

- Verificación: orientada al desarrollo del software y al uso de técnicas de desarrollo.
  - Código, diseño, testing, métricas de calidad, revisiones, …
  - Orientado a la calidad interna.
- Validación: orientada a determinar si el software cumple los requisitos del cliente.
  - Requisitos, casos de uso, aceptación, feedback del cliente, …
  - Orientado a la calidad externa.

En esta asignatura nos centramos en técnicas de verificación, aunque parte de la validación se alcanza indirectamente.

1. Motivación – Técnicas de verificación

Existen diferentes técnicas de verificación:

1. Testing.
2. Análisis estático.
3. Revisiones.
4. Pruebas formales.

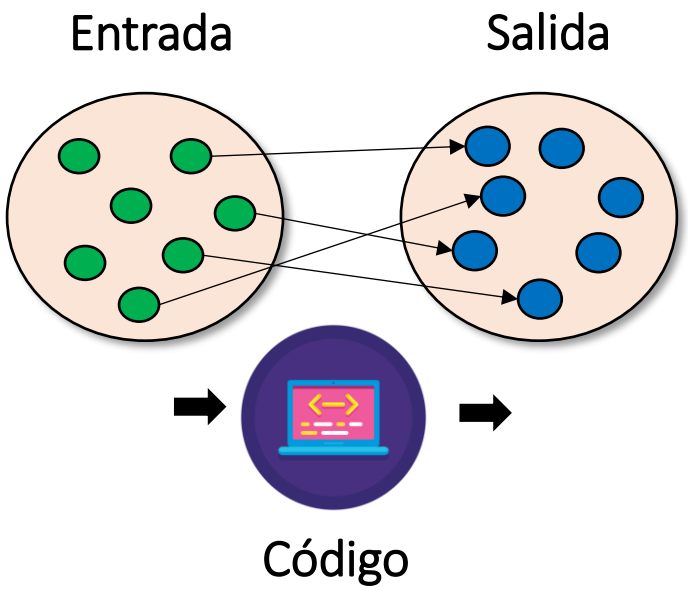
1. Motivación – Técnicas de verificación

1. Testing

- Verificación dinámica del sistema.
- El sistema se ejecutan tratando de descubrir fallos.

Ventajas e inconvenientes

- PRO: No hay falsos positivos
  si hay un fallo, es que es un fallo.
- CON: Incompleto.
  no se prueban todas las posibles ejecuciones.



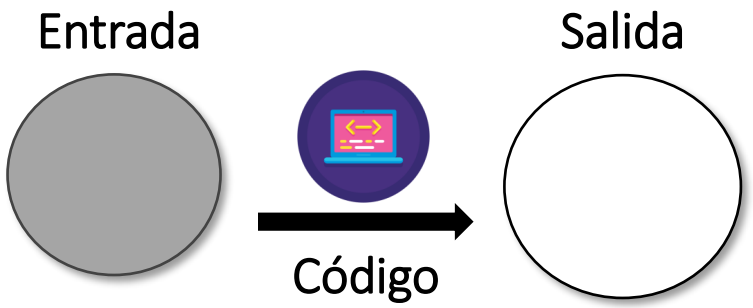
1. Motivación – Técnicas de verificación

2. Análisis estático

- Detecta problemas sin ejecutar software.
- Considera todas las posibles ejecuciones.

Ventajas e inconvenientes

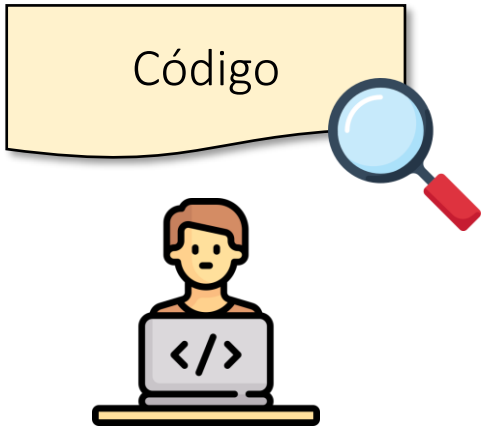
- PRO: **Considera todas** las posibles ejecuciones.
- CON: Puede generar **falsos positivos**.



1. Motivación – Técnicas de verificación

3. Revisiones

- Revisión de código y documentos.
- Revisión por otro desarrollador.



Ventajas e inconvenientes

- PRO: Sistemáticas.
  Pueden ser exhaustivas.
- CON: Informales y subjetivas.

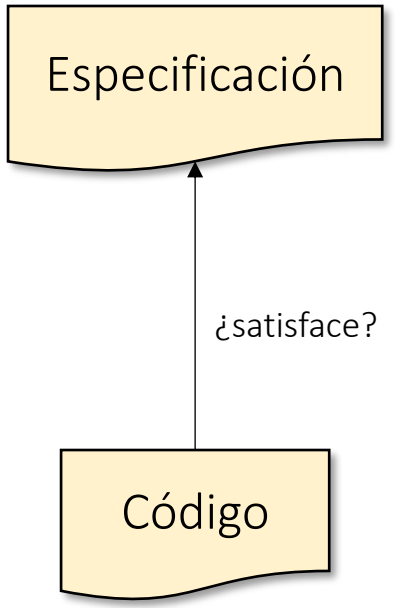
1. Motivación – Técnicas de verificación

4. Pruebas formales

- Demostración matemática.
- Cumplimiento de especificaciones.

Ventajas e inconvenientes

- PRO: Garantías fuertes de corrección.
- CON: Complejas y costosas.
  requieren escribir especificaciones.



2. Testing

1. Motivación – ¿Qué es el testing?

Testing es el proceso de ejecutar un sistema con la intención de encontrar fallos.

Proceso

- Ejecutamos el software.
- Observamos su comportamiento.
- Lo comparamos con lo esperado.

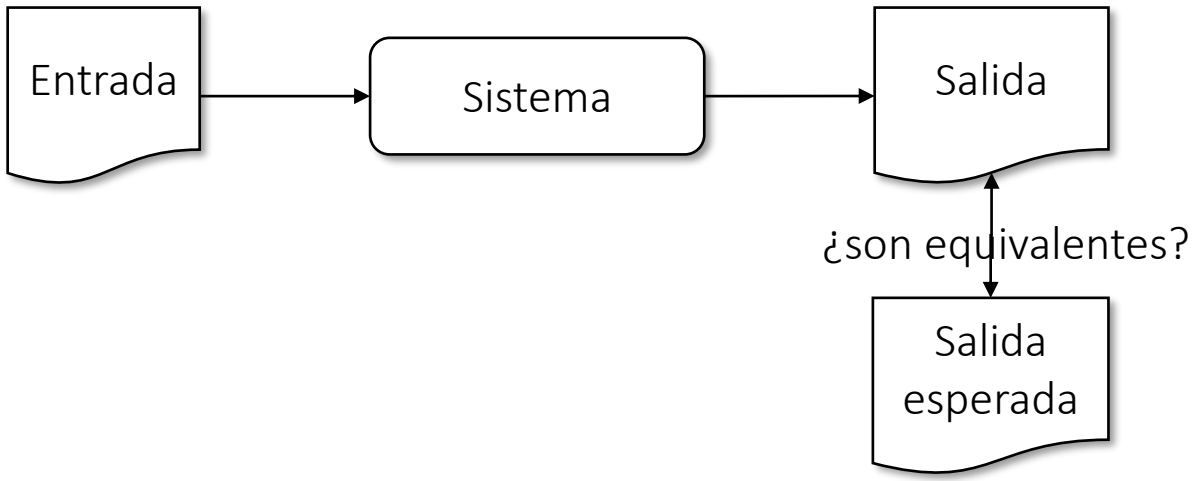
Un **test exitoso** es aquel que hace fallar el programa.

2. Testing – Motivación

En el testing estimulamos el sistema que queremos probar (System under test; SUT) mediante la ejecución de casos de prueba.

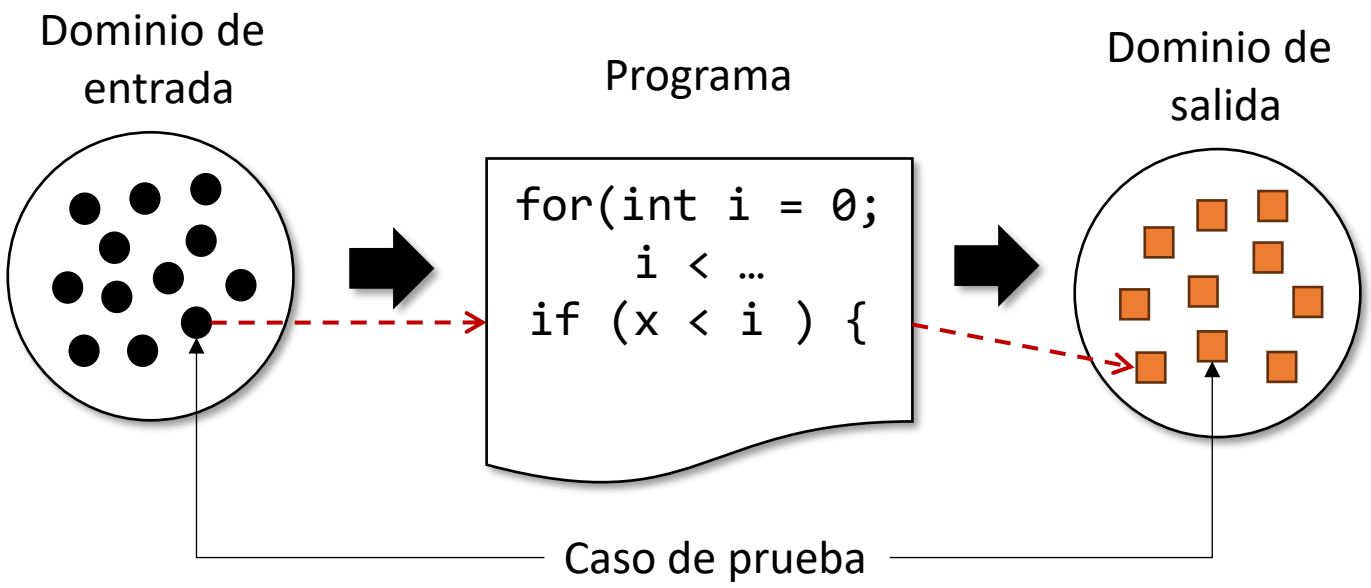
Elementos necesarios:

- **Datos de entrada**
  Datos para estimular el sistema para que produzca una salida.
- **Oráculo**
  La salida esperada o una manera de comprobar que la salida obtenida es válida.



2. Testing – Motivación

- El testing es **dinámico.**
  Es necesario ejecutar el programa para poder
- El testing es **optimista**.
  Si falla para un cierto dato, fallará para datos similares.
- El testing ejercita una (pequeña) fracción del dominio de entrada.
- Caso de prueba: `par(entrada, salida)`



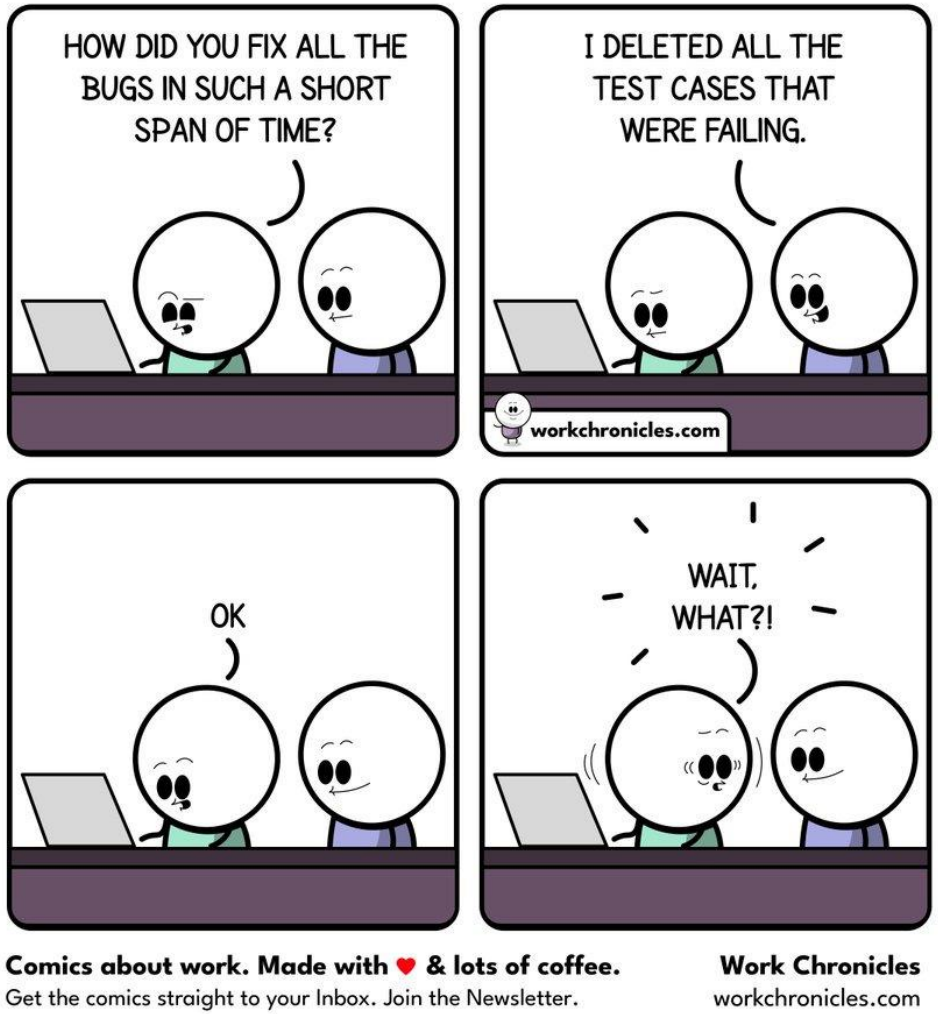
El caso de prueba detecta un error.

2. Testing – Objetivos

Un test exitoso es el que consigue hacer fallar al programa.

- El objetivo del testing no es demostrar que el software funciona.
- Es encontrar comportamientos incorrectos.
- Si un test no detecta fallos, no sabemos si el sistema es robusto.
- Un test que revela un fallo aporta información valiosa.

> “Las técnicas de testing pueden probar la existencia de errores, pero no su ausencia”
>
> Edsger Dijkstra



2. Testing – Principios

- **Presencia ≠ Ausencia.**
  Que todos los tests pasen no implica que no haya defectos.
- **Exhaustividad imposible.**
  Un formulario con 10 campos si/no tiene 2^{10} combinaciones.
- **Detectar pronto es más barato.**
  Un defecto en desarrollo más fácil de arreglar que un defecto en producción.
- **Defect Clustering.**
  La mayoría de los defectos se concentran en pocos módulos.
- **Paradoja del pesticida.**
  Si repites los mismos tests, dejan de encontrar fallos nuevos.
- **Depende del contexto**
  No se prueba igual un videojuego que un banco.
- **Falacia de la ausencia de errores**
  Ausencia de errores no implica utilidad
  Validación ≠ verificación.

* International Software Testing Qualifications Board (ISTQB)

2. Testing – Niveles

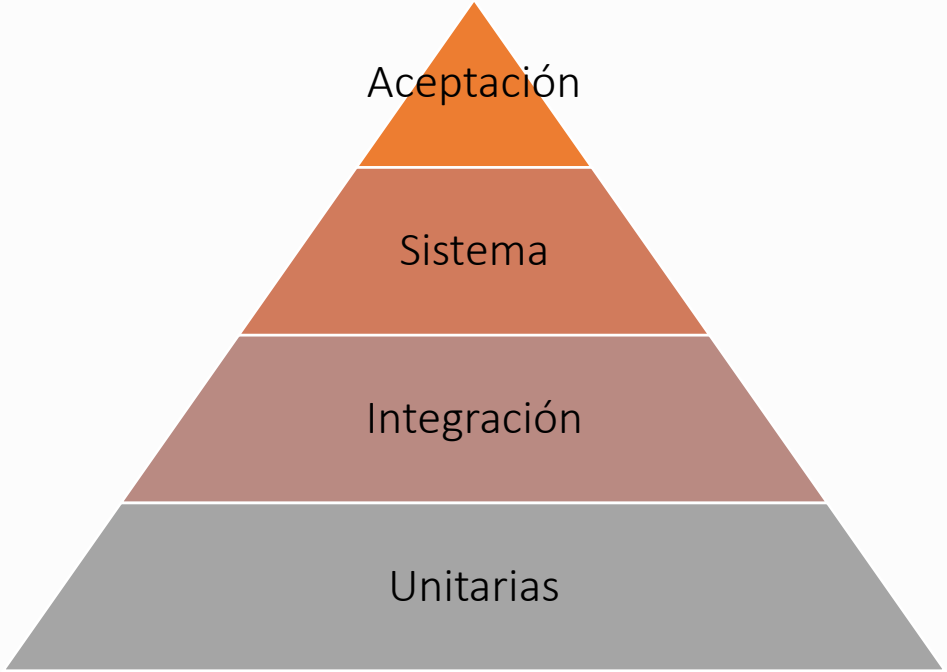
Pruebas Unitarias
Comprueban la funcionalidad de un módulo.

Pruebas de Integración
Comprueban los módulos al integrarse.

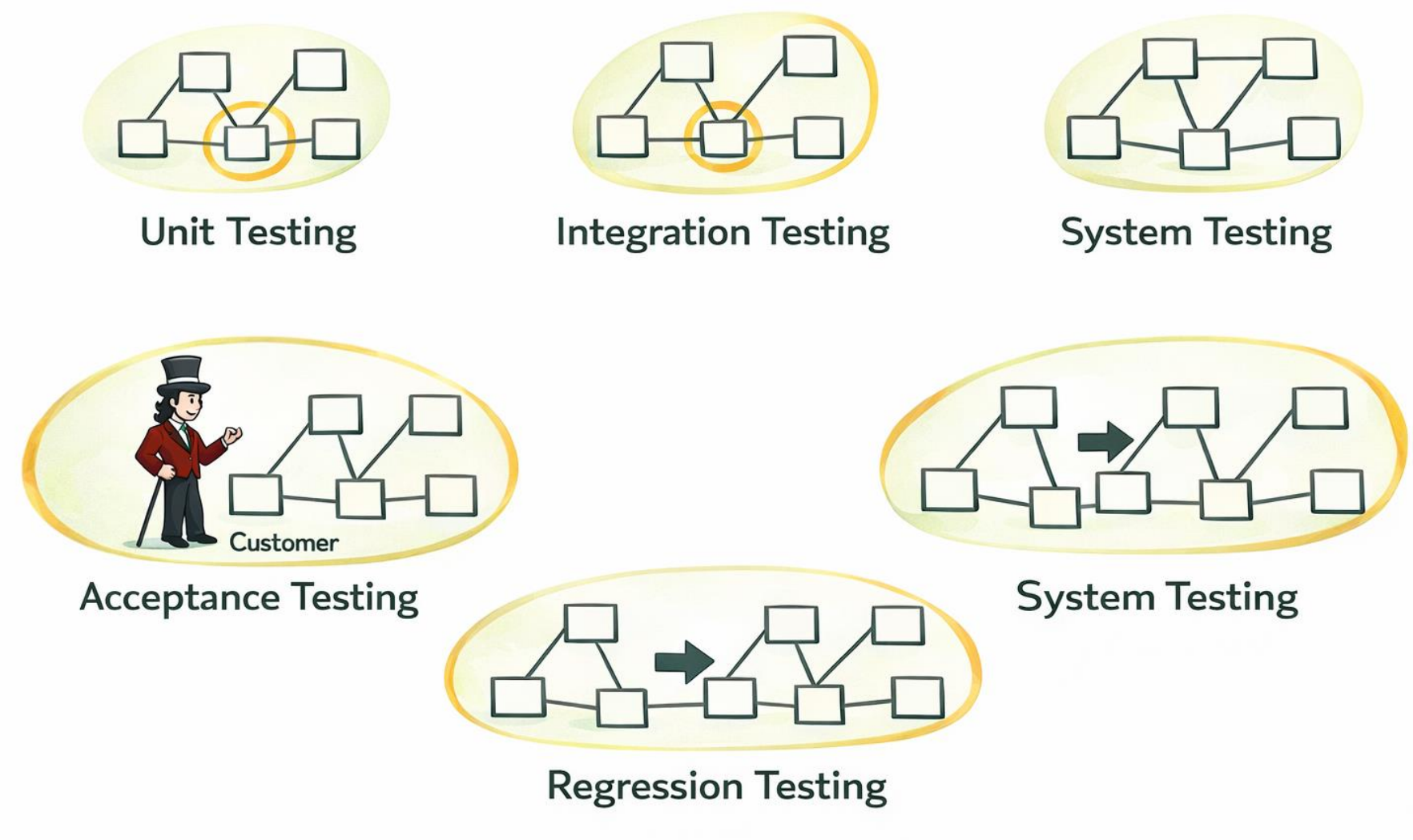
Pruebas de Sistema
comprueban requisitos no funcionales.

Pruebas de Aceptación
Comprueban requisitos del cliente.

Pruebas de regresión
Ejecutan todos los tests cuando se modifica el sistema.



2. Testing – Niveles



2. Testing – Niveles – Pruebas unitarias

Objetivo:

Probar unidades de código de manera aislada.

- Normalmente un método o una clase.
- Probar una única característica del código.
- Existen distintos frameworks (junit, phpunit, …) para casi todos los lenguajes.

Esquema básico

```
Para cada caso de prueba
  Configurar el entorno
  Ejecutar test.
  Limpiar el entorno
```

2. Testing – Niveles – Pruebas unitarias

xUnit

- Familia de frameworks para testing automatizado.
- Disponible en prácticamente todos los lenguajes.
- Popularizado por Kent Beck y Erich Gamma.
- Estructura clara para escribir tests.
  - Automatización de ejecución.
  - Asserts para comparar resultados.
  - Integración con IDE y CI/CD.

JUnit 5

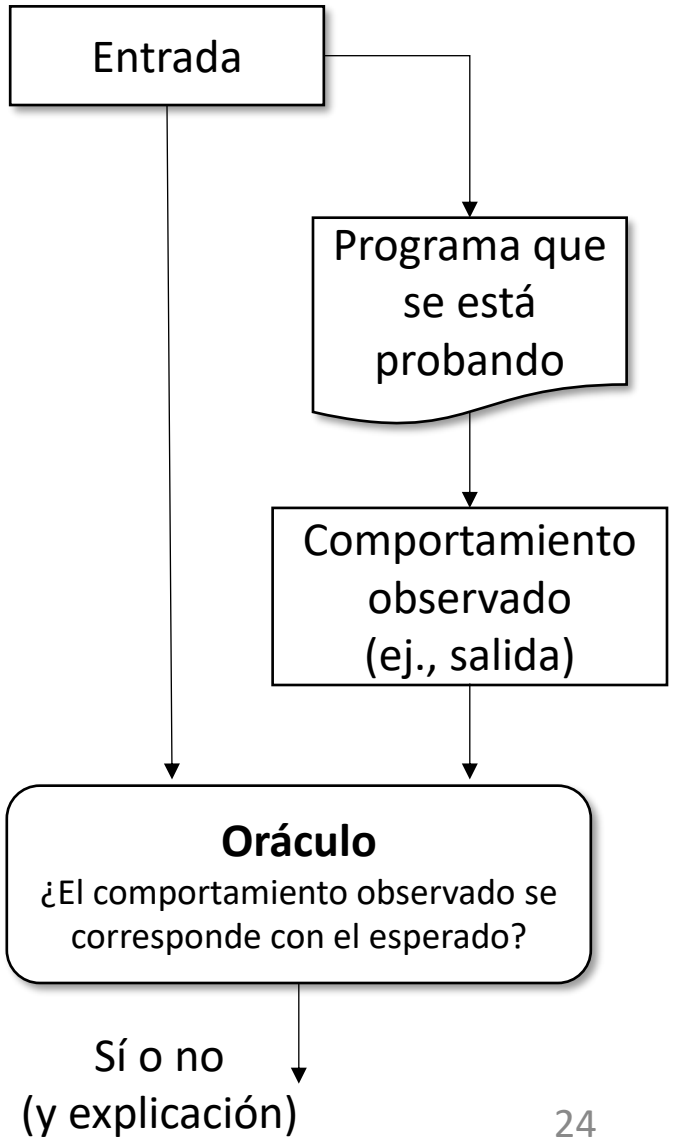
2. Testing – Niveles – Pruebas unitarias

En las pruebas de software, un **oráculo** nos ayuda a determinar si el resultado de una prueba es correcto o no.

- Puede ser tan simple como un valor de salida asociado a uno de entrada.
- Puede ser un programa diseñado para chequear el comportamiento de otros programas.

Ejemplo:

- En JUnit típicamente usamos `assert` para indicar el oráculo



2. Testing – Niveles – Pruebas de integración

Objetivo

Probar la interacción entre los módulos.

- Interés en problemas relativos a la coordinación entre módulos.
- Estas pruebas se realizan después las pruebas unitarias.

Las pruebas de integración en DDD:

- Implican probar las interacciones entre agregados, repositorios y otros servicios del dominio.
- Las pruebas de integración garantizan que sus agregados y servicios colaboren como se espera.

2. Testing – Niveles – Pruebas de integración

Estrategias de pruebas de integración

1. Todos a la vez – *"big bang integration testing"*
   - Configurar el sistema completo.
   - Se parece más a Pruebas de sistema.
   - La arquitectura hexagonal las facilita
2. Incremental.
   - Probar las interacciones poco a poco.
   - Se pueden ejercitar partes reales del sistema (*ej., una base de datos*)
   - Otras partes, se pueden configurar como *mocks.*

2. Testing – Niveles – Pruebas de integración

Ejemplo DDD

```
@Test
public void testPedidoRechazadoSiUsuarioBloqueado () {

  // Preparamos los datos y configuramos el escenario
  Usuario usuario = new Usuario("john@doe.com", EstadoUsuario.BLOQUEADO);
  Pedido pedido = new Pedido (usuario.getId(), restaurante);
  Plato pizza = new Plato (“Pizza", "pizza grande", 20.0);

  pedido.addLineaPedido (new LineaPedido (pizza, 2));

  // Arrancamos el servicio
  ServicioPedidos servicio = new ServicioPedidos();
  servicio.realizarPedido(usuario, pedido);

  // Hacemos la comprobación
  assertEquals(EstadoPedido.RECHAZADO, pedido.getEstado());
}
```

Ejemplo:
Servicio de dominio.

- Probamos la coordinación entre dos agregados.
- Supongamos un dominio con dos agregados: **`usuario`** y **`pedidos`** con la siguiente regla de negocio: “*Un usuario bloqueado no puede realizar pedidos*”.

NOTA: Se podría ver como una prueba unitaria del módulo ServicioPedidos.

2. Testing – Niveles – Pruebas de integración

```java
@Test
public void testPedidoSeRealizaCorrectamente () {

  // Configuramos el escenario
  UsuarioRepository usuarioRepo = new UsuarioRepositoryInMemory();
  PedidoRepository pedidoRepo = new PedidoRepositoryInMemory();

  Usuario usuario = new Usuario("john@doe.com", EstadoUsuario.ACTIVO);
  usuarioRepo.save(usuario);

  Pedido pedido = new Pedido(usuario.getId(), restauranteId);
  pedidoRepo.save(pedido);

  // Servicio de aplicación
  RealizarPedidoService appService = new RealizarPedidoService(usuarioRepo, pedidoRepo);

  appService.realizarPedido(usuario.getId(), pedido.getId());

  Pedido pedidoActualizado = pedidoRepo.findById(pedido.getId());

  assertEquals(EstadoPedido.CONFIRMADO, pedidoActualizado.getEstado());

}
```

Ejemplo:
Servicio de aplicación.

Se coordinan:

- Repositorios.
- Agregados.
- Reglas de negocio.

- Estamos verificando que el flujo de datos funciona.
- No dependemos de BD, red, Docker, etc.

2. Testing – Niveles – Pruebas de sistema

Pruebas *end-to-end (E2E)*

- Simulan el uso de un sistema por un usuario.
- El sistema **como un todo**: *UI* → *backend* → *BD* → *respuesta*
- Pueden ser costosas de ejecutar.
- En ocasiones, herramientas especializadas como *Selenium* o *Playwright*.

Dudas:

- ¿Cómo sería una prueba de sistema para una aplicación web?
- ¿Cómo sería una prueba de sistema para un compilador?

2. Testing – Niveles – Pruebas de sistema – Ejemplo

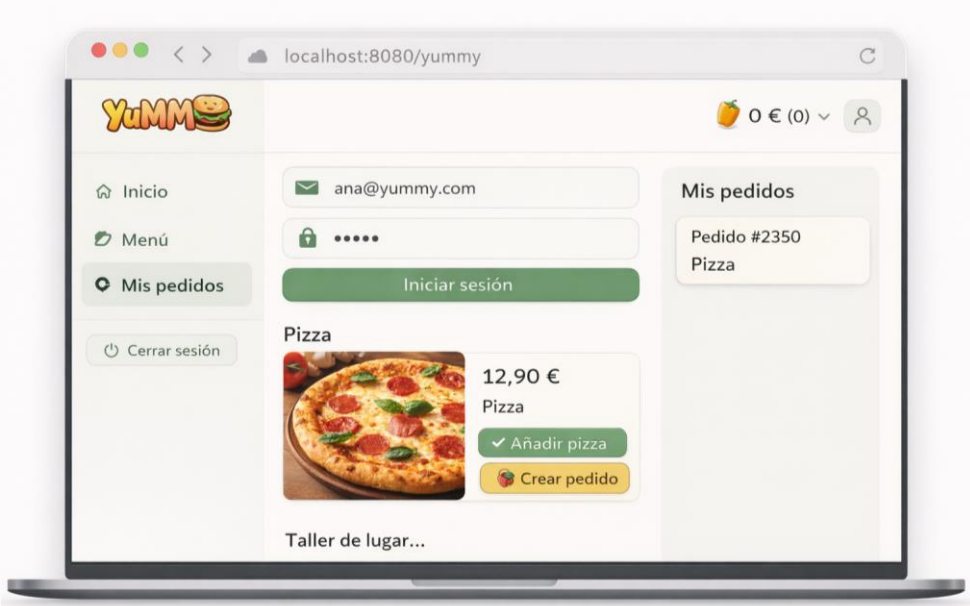
```
@Test
void login_y_crearPedido_apareceEnMisPedidos() {
  driver.get("http://localhost:8080/yummy");

  // Login
  driver.findElement(By.id("email")).sendKeys("ana@yummy.com");
  driver.findElement(By.id("password")).sendKeys("1234");
  driver.findElement(By.id("btn-login")).click();

  // Crear pedido
  driver.findElement(By.id("add-item-pizza")).click();
  driver.findElement(By.id("btn-confirmar-pedido")).click();

  // Oráculo: aparece en "Mis pedidos"
  driver.findElement(By.id("menu-mis-pedidos")).click();
  String texto = driver.findElement(By.cssSelector(".pedido:first-child")).getText();

  assertTrue(texto.contains("Pizza"));
}
```



2. Testing – Niveles – Pruebas de regresión

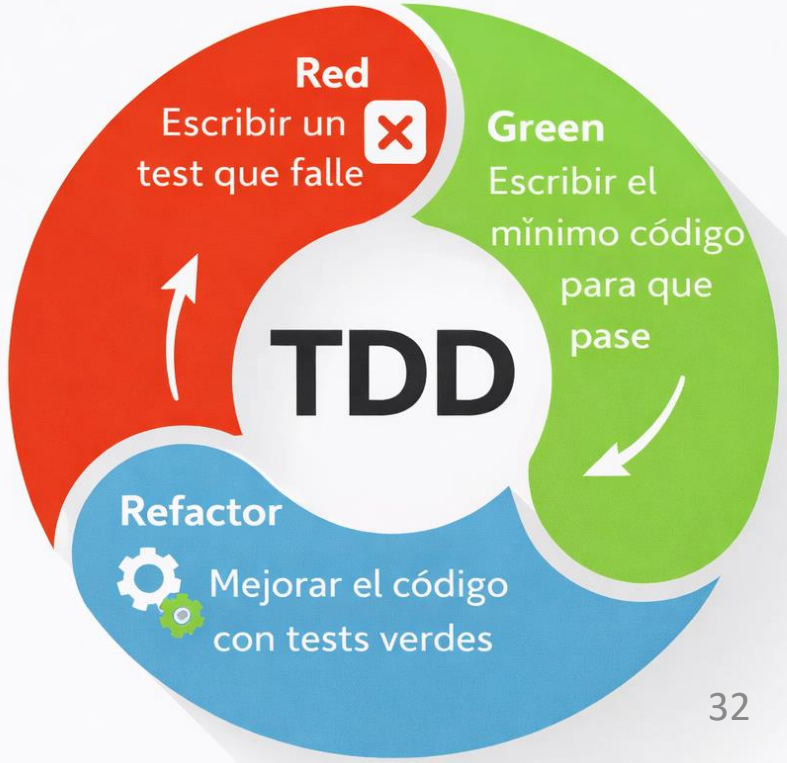
Las **pruebas de regresión** hacen referencia a la estrategia de pruebas utilizada para ejecutar las pruebas existentes cada vez que se añade nueva funcionalidad o se arregla un defecto

- Cualquier cambio en el software puede introducir problemas en otras partes
  - Especialmente si el software no está bien modularizado
- Siempre hay que ejecutar de nuevo las pruebas que tenemos
- Si el *test suite* es muy grande
  - Priorizar y seleccionar pruebas

2. Testing – Niveles – Pruebas de regresión

El TDD (*Test Driven Development*) es el desarrollo guiado por pruebas.

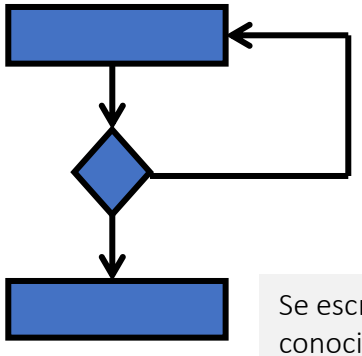
- El test se escribe antes que el código.
- Los tests especifican el comportamiento.
- Obliga a pensar en el diseño antes de implementar.
- Reduce código innecesario.
- Fomenta diseño desacoplado y testeable.
- Genera suite de regresión automáticamente.



2. Testing – Tipos de pruebas

- **Pruebas de caja negra** *(AKA: testing funcional o basado en la especificación.)*
  - Diseñadas sin conocer la implementación.
  - Basadas en especificaciones externas del software (requisitos funcionales de la aplicación, descripción de interfaces, diseño, etc.)

- **Pruebas de caja blanca** *(AKA: testing estructural.)*
  - Examina la implementación del sistema.
  - Las pruebas derivan de la estructura del código fuente a probar: bifurcaciones, iteraciones, llamadas a funciones, …



Se escriben conociendo el código, por ejemplo para asegurar que se cubren todas las líneas

3. Pruebas de caja negra

Pruebas basadas en la especificación

3. Pruebas de caja negra

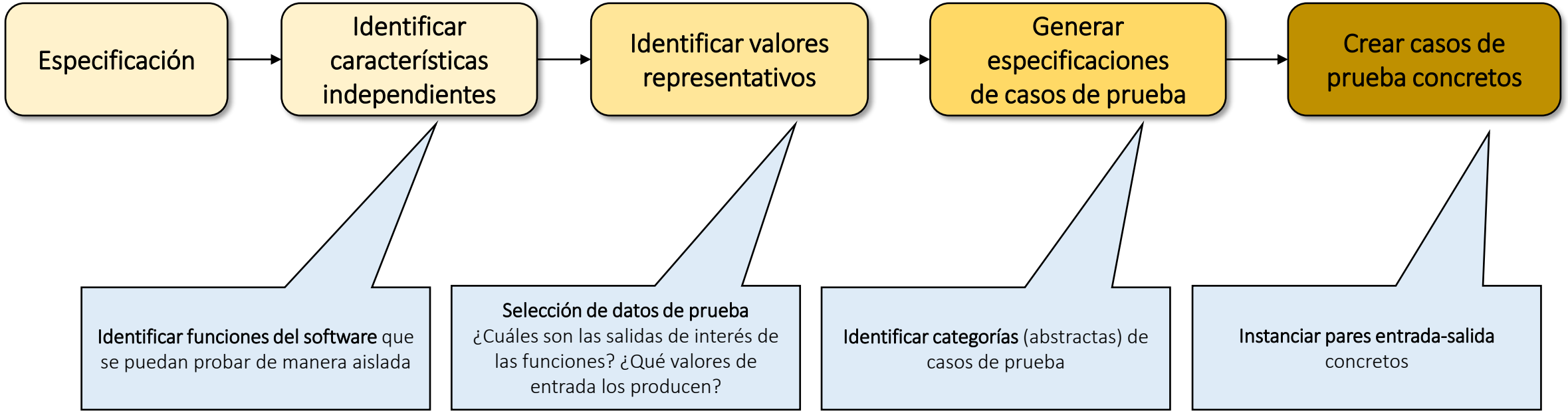
Las **pruebas de caja negra** se centran en el comportamiento del sistema, sin tener en cuenta su implementación interna.

Características

- Se toma como base la especificación.
- Centradas en el dominio del problema, no en la implementación.
  - Capturan errores lógicos.
  - Se centra en los *comportamientos importantes* del software (los que están especificados)
- No es necesario conocer el código o que esté implementado.
- Se pueden aplicar a cualquier nivel de pruebas.

3. Pruebas de caja negra – Testing funcional

Partimos de una especificación del comportamiento esperado del software (ej., *los requisitos*)



3. Pruebas de caja negra – Identificación características

La **identificación de características independientes** depende del nivel al que estemos probando:

- **Pruebas unitarias.**
  Las características se descompondrán a nivel de métodos o clases.
- **Pruebas de integración.**
  Las características de descompondrán a nivel de módulos.
- **Pruebas de sistema.**
  Interfaces de usuario o APIs.

3. Pruebas de caja negra – Selección de datos de prueba

¿Cómo seleccionamos nuestros datos de prueba?

```
public boolean aplicarDescuento(long precio)
```

Podemos usar un enfoque *naive,* probando todos los posibles valores de **precio**.

Pero… **precio** es **long**.
Tiene 2^{64} posibles valores distintos (1.8×10^{19} apróx.)

Si hiciéramos una prueba por nanosegundo, tardaríamos aproximadamente 570 años.

"Para cualquier sistema de tamaño real, el número de casos de prueba posibles es prácticamente infinito. No tenemos tiempo suficiente para ejecutarlos todos.
Es necesario seleccionar un subconjunto de casos de prueba."

* Se usa long solo para simplificar (con un double la situación es mucho más complicada)

3. Pruebas de caja negra – Selección de datos de prueba

Random testing

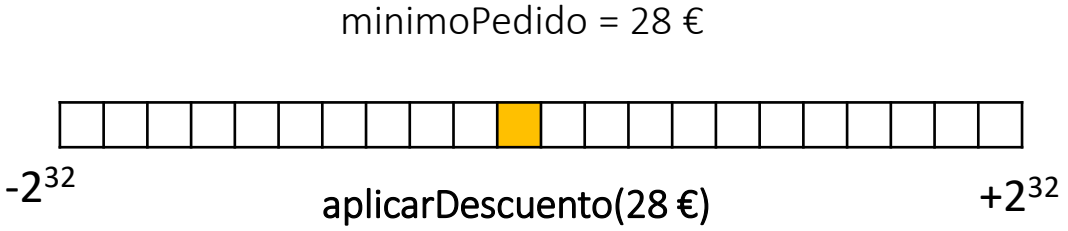
- Consiste en elegir entradas aleatoriamente
  - Cualquier entrada es "*igual de buena*" que otra para encontrar un error.
  - No hay sesgo del tester ni de malinterpretar la especificación.

Pero… **Los defectos no se distribuyen de manera uniforme.**

Especificación de ***`aplicarDescuento(precio)`***: "Se aplicará el descuento si el precio es mayor o igual que el pedido mínimo especificado en la configuración del sistema"

Si escribiste: `(precio > minimoPedido)` y no `(precio >= minimoPedido)`

¿Cuál es la posibilidad de encontrar ese bug eligiendo valores al azar?



3. Pruebas de caja negra – Partition testing

- Elegir “manualmente” entradas que sean especialmente relevantes.
- En la práctica, los errores similares se acumulan en la misma zona.

El partition testing se basa en la idea de que algunos conjuntos de entradas hacen que el programa se comporte igual independientemente de los valores concretos.

3. Pruebas de caja negra – Partition testing

Un conjunto de valores de entrada son una clase de equivalencia **(equivalence partition)** si se puede asumir de manera razonable que provocarán salidas "*similares*" al ejecutar la implementación.

Objetivo

- Definir grupos de entradas que provocan el mismo comportamiento en el programa.
  1. Leer la especificación
  2. Definir particiones
- Generación de casos de prueba concretos
  Elegir un valor de cada partición. Todos son equivalentes.

```
int abs(int valor) { … }
```

Clases de equivalencia

1. **valor < 0:** abs(-37) = 37, abs(-36) = 36, etc.
2. **valor > 0:** abs(10) = 10, abs(11) = 11, etc.
3. **valor = 0:** No es ninguna de las anteriores. abs(0) = 0

3. Pruebas de caja negra – Clases válidas e inválidas

Las entradas se pueden categorizar en:

- Clase válida: Entradas que el sistema debería aceptar y procesar correctamente.
- Clase inválida: Entradas que no cumplen los requisitos y el sistema debería rechazar con un error

Si la especificación incluye condiciones sobre la entrada *(ej., el valor tiene que ser mayor que 0)*, usar esta información para derivar las clases válidas e inválidas.

- Clase válida: dentro del rango.
- Clase inválida: fuera del rango (muy pequeño).
- Clase válida: fuera del rango (demasiado grande).

3. Pruebas de caja negra – Clases válidas e inválidas

- Las entradas inducen clases de equivalencia sobre entradas válidas e inválidas

Tipo de entrada	Clases de equivalencia
Rango	1 válida, 2 inválidas (mayor y menor)
Valor	1 válida, 2 inválidas (mayor y menor)
Miembro de un conjunto	1 válida, 1 inválida
Booleano	1 válida, 1 inválida

- Edad permitida para registrarse: 18 a 65 años
  - Clase válida: 18–65. Clase inválida (menor): < 18. Clase inválida (mayor): > 65
- Campo “Acepta términos y condiciones”
  - Clase válida: true. Clase inválida: false

3. Pruebas de caja negra – Partition testing

- **Partition testing** nos ayuda a encontrar rangos de valores que son interesantes
  - Pero no podemos elegir todos los valores de la partición
- Necesitamos elegir al menos un valor concreto de cada partición
- ¿Qué valores usar en realidad?

3. Pruebas de caja negra – Boundary value analysis

- Los errores tienden a ocurrir en los límites del dominio (*o sub-dominio*).
- Podemos seleccionar valores concretos que exploren esos límites

Especificación: los menores de 18 tienen descuento para entrar a un parque.

```
public boolean tieneDescuento(int edad)
```

Regla:

- Si edad < 18 → no tiene descuento.
- Si edad >= 18 → no tiene descuento.

En este punto el programa cambia su comportamiento.
Hay dos particiones.

- **Límite:** 18
- **Clase menor:** 17
- **Clase mayor:** 19

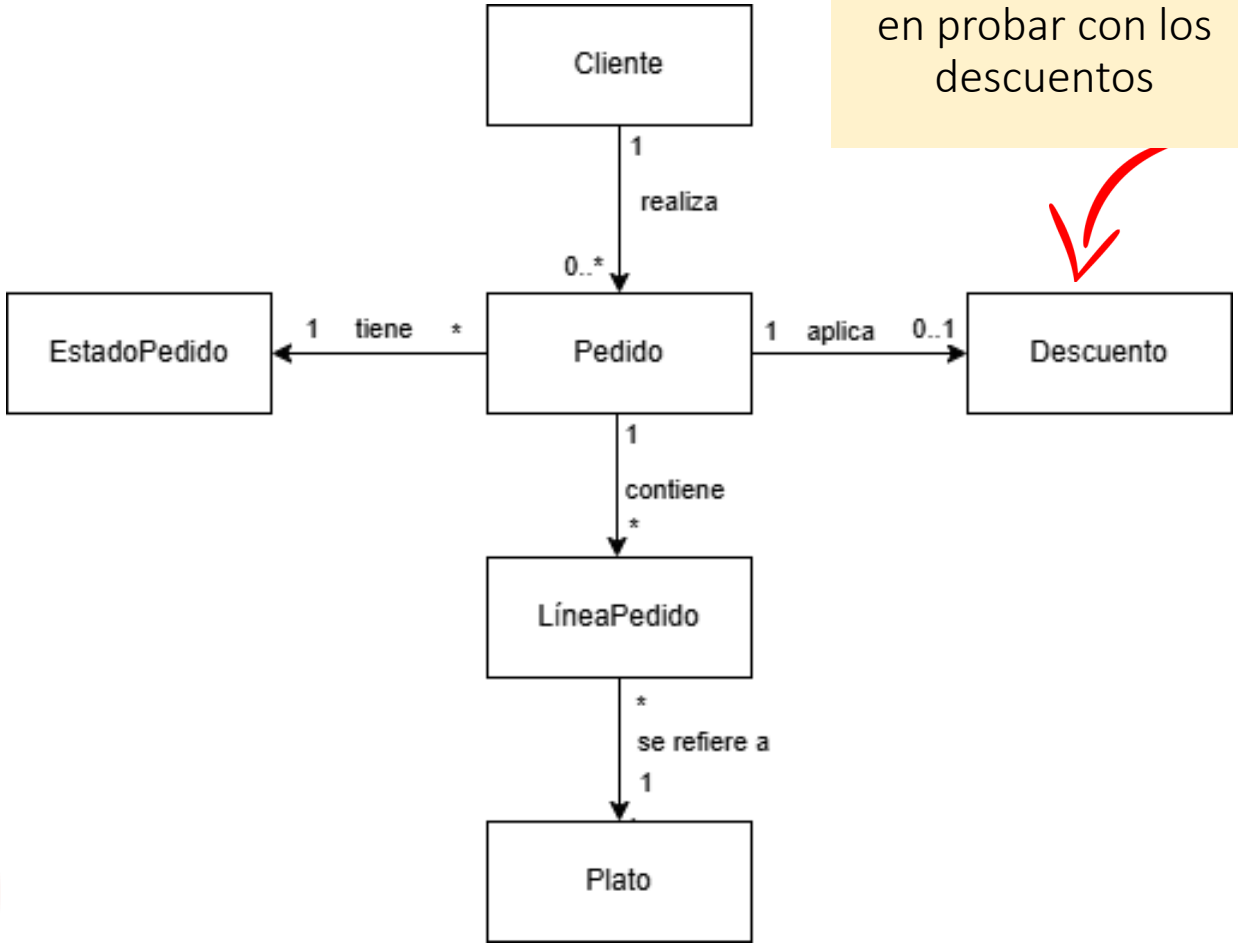
3. Pruebas de caja negra – Ejercicio

Vamos a ver esto con un caso de estudio.

La UMU quiere implantar un servicio de comida rápida y a domicilio, llamado **yUMmy**, pensado especialmente para estudiantes que pasan muchas horas en la universidad y no siempre tienen tiempo para cocinar o salir a comer.

La idea es ofrecer una aplicación sencilla y eficiente para que los estudiantes puedan hacer pedidos de comida a los restaurantes del campus o cercanos, de forma rápida, con precios especiales y descuentos para la comunidad universitaria.

Vamos a centrarnos en probar con los descuentos



3. Pruebas de caja negra – Ejercicio

¿Qué vamos a probar?:

La clase ***PoliticaDescuento*** *aplica un porcentaje de descuento a un pedido si este supera el importe mínimo. Si no se cumple la condición, devuelve el precio original.*

```
public class PoliticaDescuento {
  private double minimoPedido;
  private double porcentaje;

  public PoliticaDescuento(double minimo, double porcentaje) {
    this.minimoPedido = minimo;
    this.porcentaje = porcentaje;
  }

  /**
   * Se aplica un descuento si el precio del pedido es
   * es mayor o igual que el importe mínimo (minimoPedido).
   */
  public double aplicarDescuento (double precioBase) {
    // La implementación no es relevante
  }
}
```

3. Pruebas de caja negra – Ejercicio

testDescuentoAplicado

Verificar que cuando el precio del pedido es mayor que 20 €, el descuento se aplica correctamente.

- Descuento disponible
  10% en pedidos a partir de 20 €.
- Precio base del pedido
  25 €.
- Descuento esperado
  10% de 25 € = 2.5 €.
- Precio final esperado
  22.5 €.

```
@Test
public void testDescuentoAplicado () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.10);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (25.0);

  // (3) Valido mi prueba
  assertEquals (22.5, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testSinDescuentoAplicado

Verificar que cuando el pedido es menor que 20 €, NO se aplica ningún descuento.

- Descuento disponible
  10% en pedidos a partir de 20 €.
- Precio base del pedido
  15 €.
- Descuento esperado
  (no aplica).
- Precio final esperado
  15 €.

```
@Test
public void testSinDescuentoPorBajoMinimo () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.10);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (15.0);

  // (3) Valido mi prueba
  assertEquals (15.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testPorcentajeCero

Verificar que cuando el porcentaje de descuento es 0, no se aplica descuento.

- Descuento disponible
  0% en pedidos a partir de 20 €.
- Precio base del pedido
  30 €.
- Descuento esperado
  0 €.
- Precio final esperado
  30 €.

```
@Test
public void testPorcentajeCero () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 0.0);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (30.0);

  // (3) Valido mi prueba
  assertEquals (30.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testDescuentoTotal

Verificar que cuando el porcentaje de descuento es 100%, el pedido queda a 0 €, siempre que supere el mínimo.

- Descuento disponible
  100% en pedidos a partir de 20 €.
- Precio base del pedido
  50 €.
- Descuento esperado
  0 €.
- Precio final esperado
  30 €.

```
@Test
public void testDescuentoTotal () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (20.0, 1.0);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (50.0);

  // (3) Valido mi prueba
  assertEquals (0.0, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testSinDescuentoMinimo

Verificar que puedo tener descuentos sin precio mínimo.

- Descuento disponible
  10% en todos los pedidos.
- Precio base del pedido
  5 €.
- Descuento esperado
  10% de 5 € = 0,5 €.
- Precio final esperado
  4,5 €.

```
@Test
public void testSinDescuentoMinimo () {

  // Preparo mi caso de prueba
  Descuento d = new Descuento (0.0, 0.1);

  // (2) Ejecuto mi prueba
  double precio = d.aplicarDescuento (5.0);

  // (3) Valido mi prueba
  assertEquals (4.5, precio, 0.001);
}
```

3. Pruebas de caja negra – Ejercicio

testPorcentajeFueraDeRango

Verificar que no existan porcentajes por encima y por debajo del mínimo .

```
@Test
public void testPorcentajeFueraDeRango () {
  assertThrows (IllegalArgumentException.class, () -> new Descuento (20.0, 1.5));
  assertThrows (IllegalArgumentException.class, () -> new Descuento (20.0, -0.1));
}
```

3. Pruebas de caja negra - Ejemplo

NUEVA REGLA DE NEGOCIO

A partir de ahora, el descuento que se aplique no puede superar los 100 € por pedido, aunque el porcentaje aplicado sea muy alto.

¿Son suficientes los tests que teníamos para las nuevas reglas de negocio?

```
public class Descuento {
  …;
  private static final double MAXIMO_DESCUENTO = 100.0;

  public double aplicarDescuento (double precioBase) {
    if (precioBase >= minimoPedido) {
      double descuento = precioBase * porcentaje;
      if (descuento > MAXIMO_DESCUENTO) {
        descuento = MAXIMO_DESCUENTO;
      }
      return precioBase - descuento;
    } else {
      return precioBase;
    }
  }
}
```

3. Pruebas de caja negra

Reflexiones y estrategias

- Tiene sentido comprobar un valor representativo y los valores límite.
- Los valores representativos (particiones de equivalencia) nos ayudan a ver como se comporta el sistema con valores típicos.
- Siempre interesa al menos tener una prueba de frontera para cada valor.
- También añadir pruebas con valores límite.